Computer Vision

Segmentation

Keunsoo Ko

The Catholic University of Korea
Department of Artificial Intelligence

가톨릭대학교
THE CATHOLIC UNIVERSITY OF KOREA

```python
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from skimage.segmentation import slic, mark_boundaries

# 1. Image Normalization
pil_image = Image.open("Bear.png").convert("RGB")
pil_image.thumbnail((480, 480))
image = np.array(pil_image) / 255.0

# Apply SLIC
segments = slic(image, n_segments=200, compactness=10,
                sigma=1, start_label=0, convert2lab=True)

h, w = segments.shape
seed = segments[h // 2, w // 2]
n = segments.max() + 1

plt.figure(figsize=(6, 5))
plt.imshow(mark_boundaries(image, segments)) # Visualizae the boundaries of the segments
plt.scatter(w // 2, h // 2, c="cyan", s=30) # Center circle we will use
plt.show()
```

```python
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from skimage.segmentation import slic, mark_boundaries

# 1. Image Normalization
pil_image = Image.open("Bear.png").convert("RGB")
pil_image.thumbnail((480, 480))
image = np.array(pil_image) / 255.0

#
se                                    compactness=10,
                                      convert2lab=True)

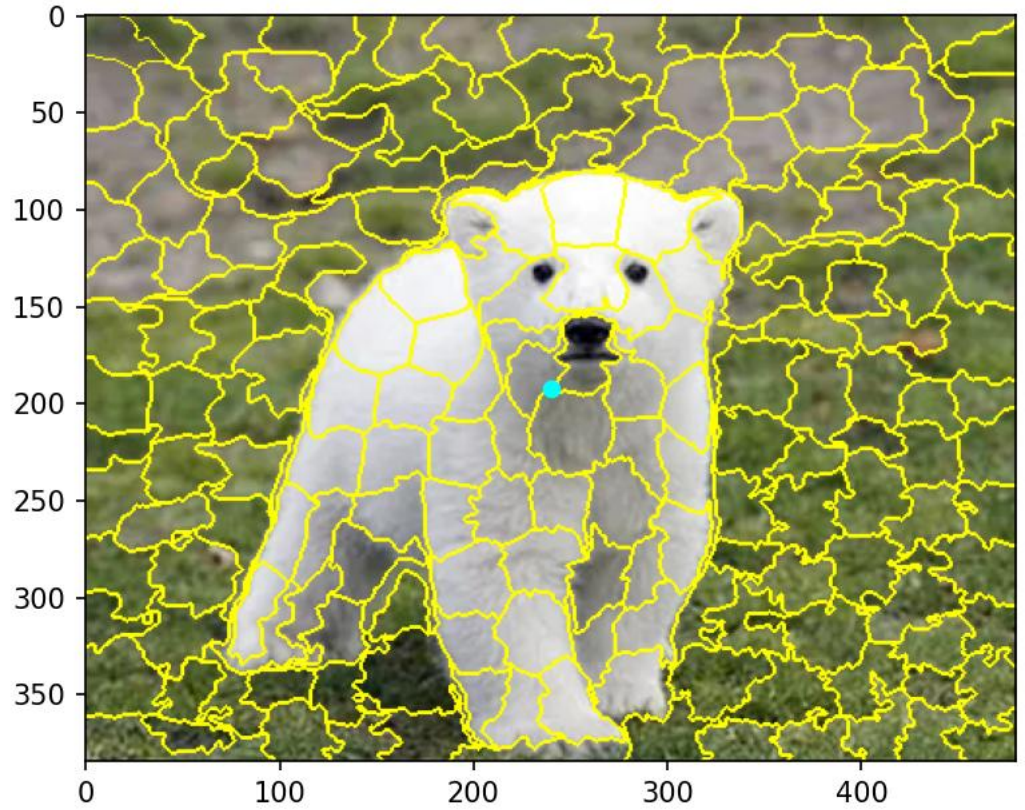
h,
se
n

pl
pl                                    nts)) # Visualize
pl                                    =30) # Center circ
plt.show()
```

```
print(segments)
```

```
[[  0   0   0 ...  12  12  12]
 [  0   0   0 ...  12  12  12]
 [  0   0   0 ...  12  12  12]
 ...
 [186 186 186 ... 185 185 185]
 [186 186 186 ... 185 185 185]
 [186 186 186 ... 185 185 185]]
```





```
super_pixels = np.zeros((n, 3))

for i in range(n):
    super_pixels[i] = image[segments == i].mean(axis=0)

print("# of superpixel:", n)
print("The super-piexl of the image center:", np.round(super_pixels[seed], 3))

# Here, RGB features are used,
# But you can also use other features like LAB.
features = super_pixels

mask = (segments == seed)

from scipy.ndimage import binary_dilation

border = binary_dilation(mask) & ~mask
neighbors = np.unique(segments[border])
```

```
super_pix

for i in
    super

print("#
print("Th                                                    d], 3))

# Here, F
# But you
features

mask = (segments == seed)

from scipy.ndimage import binary_dilation

border = binary_dilation(mask) & ~mask
neighbors = np.unique(segments[border])
```



```
features = super_pixels

mask = (segments == seed)

from scipy.ndimage import binary_dilation

border = binary_dilation(mask) & ~mask
neighbors = np.unique(segments[border])

threshold = 0.5 # Let's find the optimal threshold for your image.
for i in neighbors:
    difference = np.linalg.norm(features[i] - features[seed])
    if difference < threshold:
        mask[segments == i] = True
```

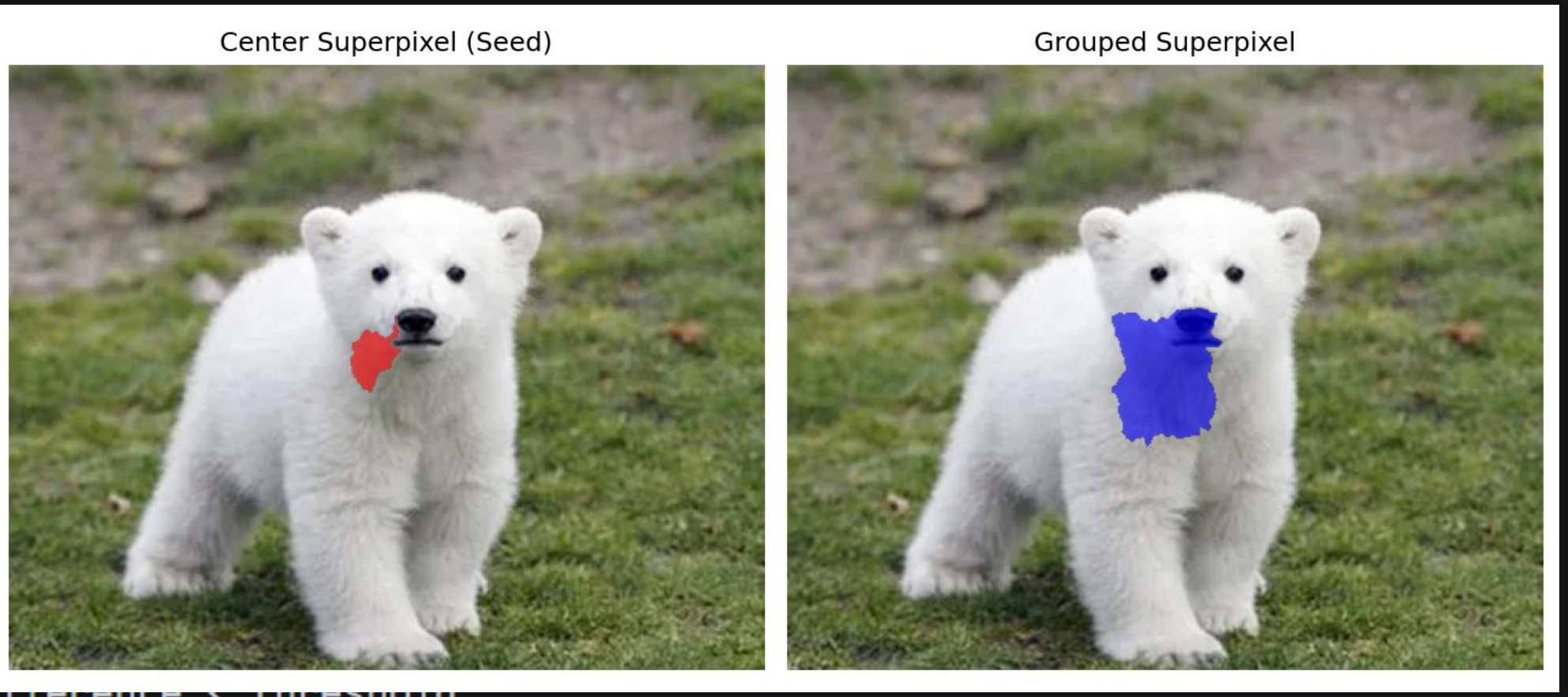
```
features = super_pixels

mask = (s

from sci

border =
neighbors

threshold
for i in
    diffe
    if di
        mask[segments == i] = True
```



```
features = super_pixels

mask = (segments == seed)

from scipy.ndimage import binary_dilation

border = binary_dilation(mask) & ~mask
neighbors = np.unique(segments[border])

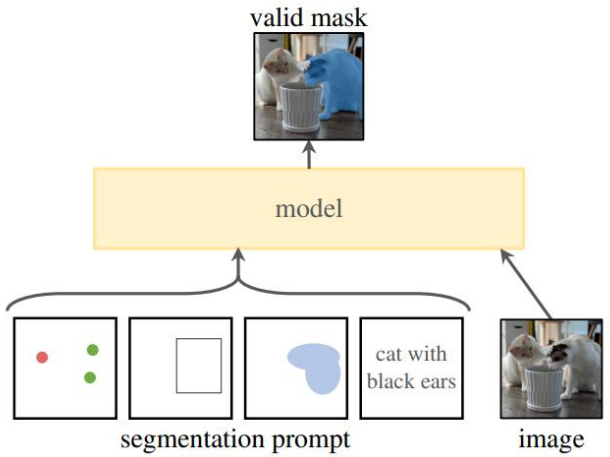
threshold = 0.5 # Let's find the optimal threshold for your image.
for i in neighbors:
    difference = np.linalg.norm(features[i] - features[seed])
    if difference < threshold:
        mask[segments == i] = True
```

Repeat this process with an appropriate threshold!!

We can segment the object →



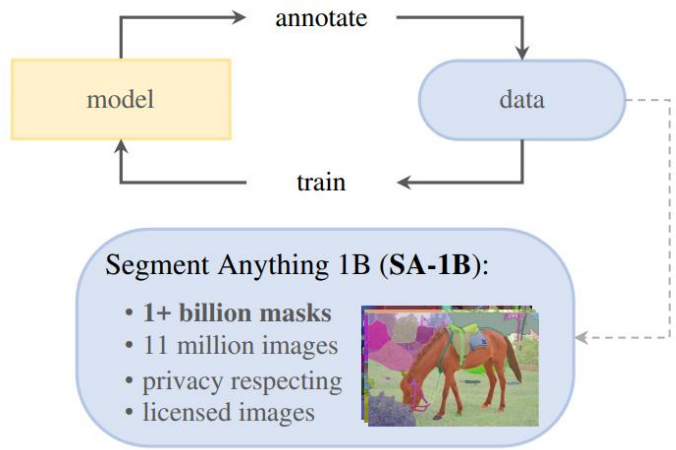
SAM



(a) **Task**: promptable segmentation



(b) **Model**: Segment Anything Model (**SAM**)



(c) **Data**: data engine (top) & dataset (bottom)

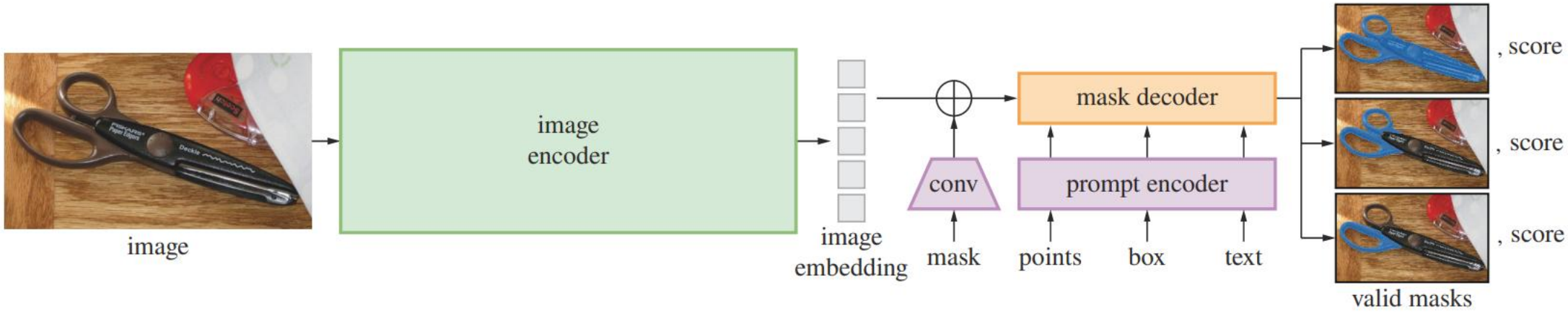


Figure 4: Segment Anything Model (SAM) overview. A heavyweight image encoder outputs an image embedding that can then be efficiently queried by a variety of input prompts to produce object masks at amortized real-time speed. For ambiguous prompts corresponding to more than one object, SAM can output multiple valid masks and associated confidence scores.

```python
from ultralytics import SAM
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt

# Image
path = "Bear.png"
image = np.array(Image.open(path).convert("RGB"))
h, w = image.shape[:2]

# SAM 2.1
model = SAM("sam2.1_t.pt")

# Segmentation based on center point
results = model(path, points=[w//2, h//2], labels=[1])

# Predicted mask
mask = results[0].masks.data[0].cpu().numpy().astype(bool)
```

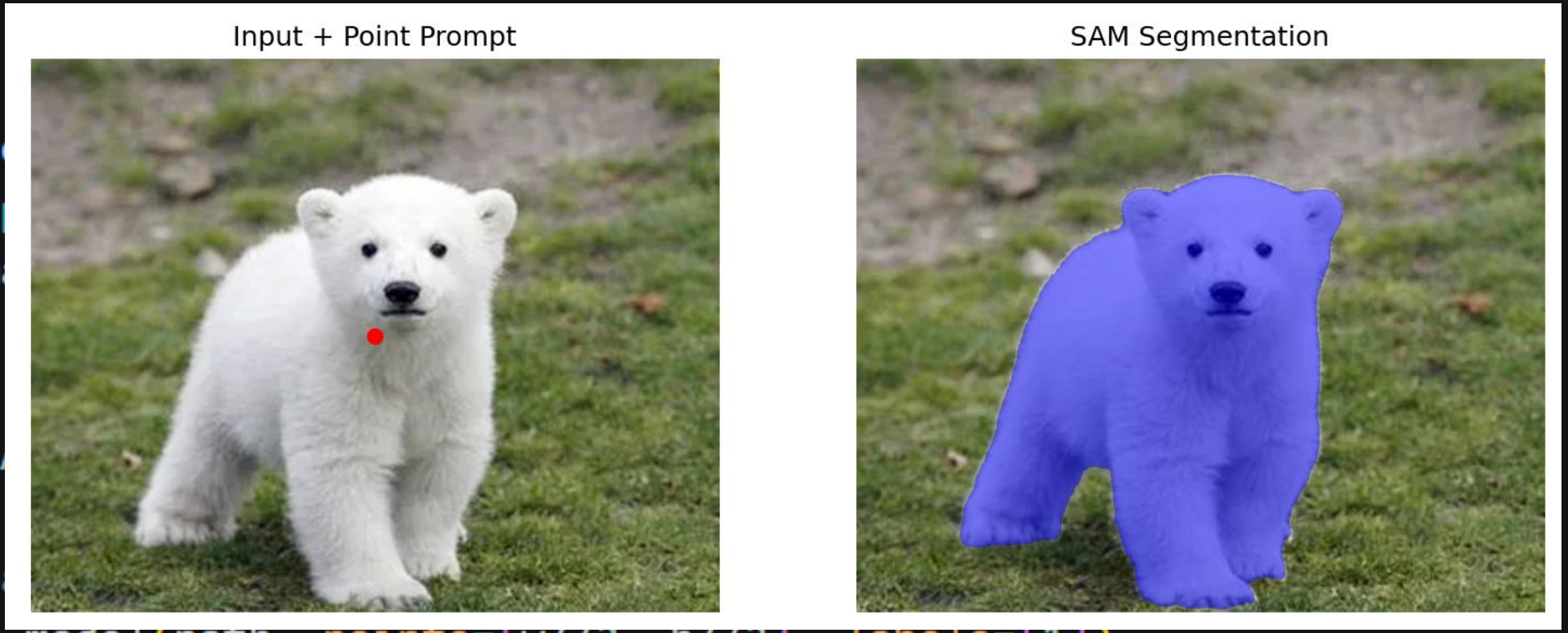
```python
from ultralytics import SAM
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt

# Image
path = "B
image = n
h, w = im

# SAM 2.1
model = S

# Segment
results = model(path, points=[w//2, h//2], labels=[1])

# Predicted mask
mask = results[0].masks.data[0].cpu().numpy().astype(bool)
```



ToDo

- Segment your image using the superpixel
  - You need to set the image feature (RGB, LAB, or others..)
  - You need to set the appropriate threshold..!
- Segment your image using SAM